JLPT AI App, Team Project Brief

Oct 8, 2026 · @Htet Pai

How to use this brief

Copy the shared sections (everything except Role briefs) plus your own role into your AI chat, then ask it to help you build your part step by step.

Suggested opening message for your AI:

> I'm a beginner working in a 5-person student team. Below is our project brief and my role. Please explain each step simply, write code that fits our folder structure and rules, and tell me what to commit and when. Don't change files outside my area without asking me first.

When your AI suggests something that conflicts with this brief (a different framework, a new folder), don't follow it alone. Raise it with the team first.

What we are building

We are building a web and mobile app (iOS, Android and website from one codebase) that helps Burmese speakers prepare for the JLPT (N1 and N2), with AI that explains answers and plans each user's study schedule.

The app teaches vocabulary and grammar in small units, tests the user with quizzes and mock tests, and uses AI to explain mistakes and build a daily study plan based on the user's exam date.

This is a learning project. Our goal is team experience (Git, GitHub, Docker, code review), not a production launch. Working, simple code beats perfect code.

Features

These are the features of the app, in the order a user meets them.

1. **Login**: the user signs in with their Google account. No passwords stored by us. Google only confirms who the user is; our own PostgreSQL database stores the user's account, progress and results.

2. **First-time setup (onboarding)**: shown only on the first login. The user enters:
    a. Last JLPT level taken: N1, N2, or "Haven't taken it yet"
    b. Score: a whole number from 0 to 180. Anything else shows "Invalid input".
    c. Exam date: picked from a list of official JLPT dates, or "Not decided".
    d. The same fields can be changed later in Settings.
3. **Study plan**: the app calculates how many units the user should do per day before the exam. The code does the math, the AI adds friendly advice on top.
4. **Dashboard**: shows 20 sessions. A progress bar at the top shows overall completion, next to a pass prediction (合格予測, gōkaku yosoku) based on mock test results.
5. **Sessions and units**: each session has 10 units. Each unit has 20 vocabulary words and 1 grammar pattern. Each item is taught first (meaning in Burmese plus 2 to 3 example sentences), and only then practiced in the quiz. Units only load when the user opens a session, and words only load when the user opens a unit.
6. **Quiz**: after every 5 units. Each answer shows green (right) or red (wrong) immediately, with an explanation under each option.
7. **Mock test**: after each session. Follows the real JLPT format for the user's level, with a time limit. No feedback until the end, then a full review screen with red, green and explanations.
8. **AI assistant**: an "Ask AI" button under explanations, so users can ask follow-up questions about a question.
9. **Notifications**: a reminder on the user's phone if they miss their daily quota, plus one Japanese proverb (ことわざ, kotowaza) per day with its meaning.
10. **Languages**: the app's text is in Burmese and Japanese only. Burmese must use Unicode, never Zawgyi.

Tools we use

Everyone uses the same tools, so code written on one laptop runs on every laptop.

Tool	What it does in our project	Simple way to think about it
GitHub	Stores our code, tracks tasks, reviews changes	The shared office
Git	Saves versions of the code on your laptop	Save points in a video game

Tool	What it does in our project	Simple way to think about it
Docker and Docker Compose	Runs the whole app the same way on every laptop	A lunchbox with everything packed inside
Expo (React Native)	The frontend: one codebase for the iOS app, Android app and website	The shop window
FastAPI (Python)	The backend: logic, data, AI calls	The kitchen behind the counter
PostgreSQL	The database: users, progress, words, questions	The storage room
Google Sign-In	Login with a Google account	Showing your passport instead of making a new ID card
An LLM API (to be decided)	AI explanations, chatbot, study advice	The tutor

The frontend never talks to the database or the AI directly. It always asks the backend, and the backend talks to the database and the AI.

How the project is organized

Everything lives in one GitHub repository called jlpt-app, and each folder has one owner so we rarely edit the same files.

Folder	What goes inside	Owner
frontend/	All screens and UI parts (login, onboarding, dashboard, quiz, mock test, settings)	Frontend A and B
backend/ (routes and logic)	The API the frontend calls, login checks, study plan calculation, notifications	Backend
backend/ (database tables)	How users, sessions, units, words, questions and results are stored	Data and content
backend/ (AI part)	Explanation generation, the chatbot, AI study advice	AI

Folder	What goes inside	Owner
data/	Vocabulary, grammar, proverbs and mock test templates as plain files	Data and content
docs/	The API contract (which requests exist and what they return), diagrams	Everyone, reviewed by Backend
Root files	docker-compose.yml, .gitignore, .env.example, README.md	Team lead

To start the whole app, a member runs one command: docker compose up. This starts the frontend, backend and database together.

Secrets (API keys, passwords) go in a file called .env that is never uploaded to GitHub. The file .env.example shows which keys are needed, with fake values.

Team rules

The main branch must always work, so nobody changes it directly. Every change goes through a pull request and one review.

The workflow for every task:

1. Pick a task (an Issue) from the GitHub Project board and move it to In progress.
2. Create a new branch from main, named after the task, for example feature/onboarding-form or fix/quiz-colors.
3. Make small commits with clear messages, for example "Add score validation to onboarding form".
4. Push your branch and open a pull request. Link it to the Issue and describe what you changed in 2 or 3 sentences.
5. One teammate reviews it. Fix their comments, then merge.
6. Move the Issue to Done.

Other rules:

- Never commit .env or any API key. If it happens by accident, tell the team immediately.
- Pull the latest main before starting new work, so you don't build on old code.
- If a merged change breaks the app, undo it with git revert instead of deleting history.
- Stay inside your own folder. If you need a change in someone else's area, open an Issue or ask them.

- Commit with the email linked to your GitHub account, so your work shows on your profile.
- Weekly 15-minute meeting: what I did, what I'll do next, what's blocking me.

Role briefs

Each member owns one area. Names will be filled in after we assign roles.

Role 1, Frontend A (Dashboard and navigation)

Owner: (name)

You build the screens users move through after login.

- Dashboard with 20 session cards and the progress bar at the top
- Pass prediction display next to the progress bar
- Session screen showing 10 units, unit screen showing 20 words and 1 grammar pattern
- A "Continue where you left off" button
- Start with fake data, then connect to the backend once its API is ready

Role 2, Frontend B (Forms, quiz and mock test screens)

Owner: (name)

You build the screens where users type and answer.

- Login screen with the Google button
- First-time setup form with input checks (score 0 to 180, whole numbers only)
- Quiz screen with instant green and red feedback and explanations under each option
- Mock test screen with a timer and a review screen at the end
- Burmese and Japanese text for all screens

Role 3, Backend (API, login, study plan)

Owner: (name)

You build the server that every screen talks to.

- Google login check and user accounts
- The API: requests for sessions, units, quizzes, mock tests, results and settings
- Study plan calculation: units left divided by days left

- Phone notifications for missed quotas
- Write and maintain the API contract in docs/ so the frontend knows what to expect

Role 4, Data and content (Database and learning material)

Owner: (name)

You design how data is stored and prepare the learning content.

- Database tables for users, sessions, units, words, grammar, questions and results
- Vocabulary and grammar files in data/, from a source list we agree on. Each item has its Burmese meaning and 2 to 3 example sentences, AI-drafted or self-written, always checked by a person
- Mock test templates per level (question types, counts, time limits)
- The proverb list with meanings in Burmese, checked by a person

Role 5, AI (Explanations, chatbot, study advice)

Owner: (name)

You build everything that uses the AI.

- A script that generates explanations for every answer option ahead of time, so the app doesn't call the AI on every click
- The "Ask AI" chatbot, which answers using our own question and grammar data
- Friendly study advice on top of the backend's calculated plan
- Test how well the AI writes Burmese, and report problems early

Rotating roles (change every month)

- **Scrum master**: runs the weekly meeting and keeps the Project board up to date
- **Review pairs**: who reviews whose pull requests, so everyone reads code outside their own area

Review process

Every pull request gets one review before it is merged, and the reviewer checks the same short list each time.

- [ ] It does what the linked Issue asked, nothing more
- [ ] The app still starts with docker compose up

- [ ] Files changed are inside the author's own area
- [ ] No API keys, passwords or .env files are included
- [ ] Names and messages are clear enough for a teammate to understand
- [ ] The author can explain the code in their own words, even if their AI wrote it

The last point matters most. Using AI is fine, but every member must understand what they commit. If you can't explain it, ask your AI to explain it until you can.